DATA STRUCTURE AND APPLICATIONS

Data structure (D.S):

Structured representation of data is called Data structure.

(or)

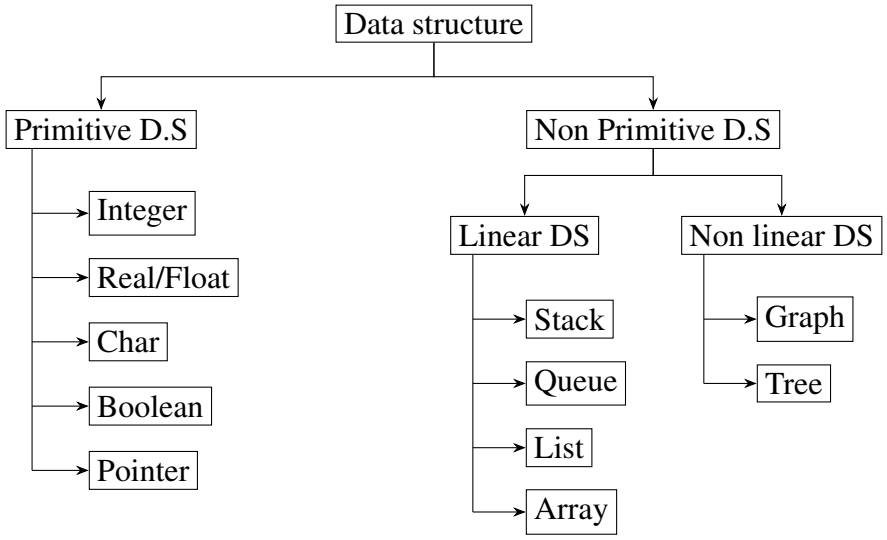
The logical (or) mathematic representation of particular organization of data is called Data structure.

Classification of D.S.

DS is normally divided into two categories

- Primitive D.S
- Non-Primitive D.S.

Below figure shows detailed classification of D.S.



DS are generally classified into primitive and non-primitive data structures. Basic data type such as int, float, character, boolean and pointers are known as primitive data structures. These data types consists of number (or) characters that can not be divided and hence they are also called simple data types.

Based on the structure and arrangement of data, non-primitive data structures are further classified into

- linear DS and
- Non-linear D.S.

A DS is said to be linear if its elements form a sequence (or) a linear list. In linear data structure, the data is arranged in a linear fashion although the way they are stored in memory need not be sequential. Arrays, stacks, Queues and linked list are examples of linear D.S.

A DS is said to be non-linear if the data is not arranged in sequence. ie. the data is arranged graphically (or) hierarchically in case of non-linear DS. Examples are tree and graph.

Arrays:

The simplest type of D.S is a linear (1-D) arrays. By 1-D array, we mean a list of a finite number n of similar data elements referenced respectively by a set of n consecutive numbers, usually 1, 2, 3, . . . , n. If we choose the name A for the array, then the element of A are denoted by subscript notation: a_1, a_2, a_3, . . . a_n

(or) Bracket notation: a[1], a[2], a[3], . . . a[n].

Ex: a[i], where i is called subscript
a is called array name
a[i] is called subscripted value.

Ex: a[10] with 10 value, 11, 3, 44, 5, 07, 15, 20, 8, 3 & 55.
ie a[0]:11, a[1]:3, a[2]:44, a[3]:5, a[4]:07, a[5]:15
a[6]:20, a[7]:8, a[8]:3 and a[9]:55.

A two-dimensional (2-D) array is a collection of similar data elements where each elements is referenced by two subscripts, namely Row and column. Such arrays are called matrices in mathematics and tables in business applications.

Ex mat[i][j] where i is called row subscript
j is called column subscript
mat is called array name
mat[i][j] is called subscripted value at i^{th} row, j^{th} col

a[3][2] ⟶3 Rows and 2 cols array is shown as:

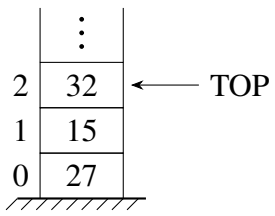
$$\begin{array}{rc} & \begin{matrix} \text{a[0][0]} & \text{a[0][1]} \end{matrix} \\ 0^{th}\text{ Row} & \begin{bmatrix} \text{a[1][0]} & \text{a[1][1]} \\ \text{a[2][0]} & \text{a[2][1]} \end{bmatrix} \\ 1^{st}\text{ Row} & \\ 2^{nd}\text{ Row} & \end{array} \quad 3\times 2$$

0th col 1st col.

Different operations performed on arrays are:
i. Create, ii. display, iii store a value
iii Retrieve a value iv. Searching, xi. Sorting.
xii. Updating a value, xiii. Merge two arrays, etc. . .

Stack

A stack, also called a Last-in-First-out (LIFO) system, is a linear list (array) in which insertions and deletions can take place only at one end, called the top. This system/structure is similar to a



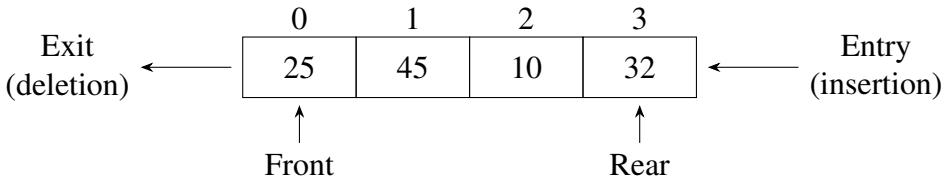
stack of dishes on a spring system. Here new dishes (or) plates are inserted only at the top of the stack and dishes/plates can be deleted only from the top of the stack. So it is called LIFO system.

Operation performed on stack are

– Insertion – PUSH operation
– deletion – POP operation and
– display

Queues:

A queue, also called a first-in-first-out (FIFO) system, is linear list (array) in which deletions can take place only at one end of the queue called front and insertions can take place only at the other end of the queue called rear end.

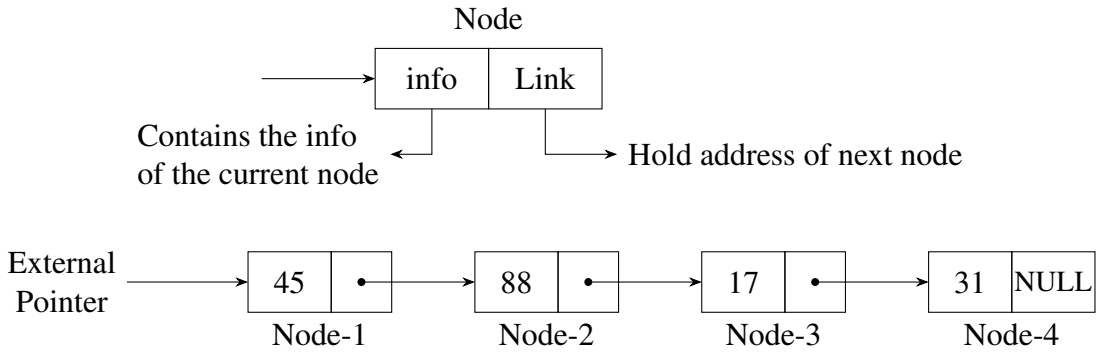


This DS operates in much the same way as a line of people waiting at ATM machine, the first person in line is the first person to used exit ATM machine.

Operation performed on Queue are i. Insertion
ii Deletion and iii. Display.

Linked list:

A linked list (or) one-way list, is a linear collection of data elements called nodes; where the linear order is given by means of pointers. ie. each node is divided into two parts: the first part contains the information of the element and second part called the Link field (or) next-pointer field (or) address-of-next node, contains the address of next node in the list.



Above figure is a schematic diagram of a linked list with 4 nodes. Each node is pictured with two parts. The left part represents the information part of the node, which may contains an entire record of data item/node. The right part represents the link (or) next pointer field of the node, and there is an arrow (⟶) drawn from it to the next node in the list. This follows the usual practice of drawing an arrow from a field to a node when the address of the node appears in the given field. The pointer of the last node contains a special value, called the NULL pointer, which indicates end of linked list.

Operation performed on linked list are

i. insert from front of the linked list

ii insert from end of the linked list

iii insert between two nodes in the linked list.

iv. delete a node from front of linked list

v. delete a node from end of linked list

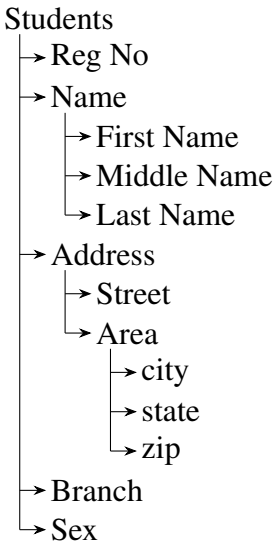
vii. delete a node between (or) i^{th} node.

xiii. Display/traverse linked list. etc...

Trees:

Data frequently contains a hierarchical relationship between various elements. The D.S which reflects this relationship is called a rooted tree graph (or) in simple a tree.

Example: A student record can described by means of a tree structure. Student record contains:



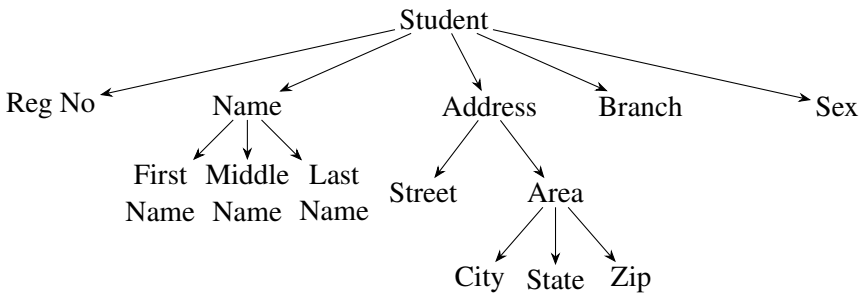


Fig: Tree representation of student record.

Graphs:

Data sometimes contains a relationship between pairs of elements which is may (or) may not necessarily hierarchical in nature. For example a person travels only between the cities connected by lines is shown in below figure.

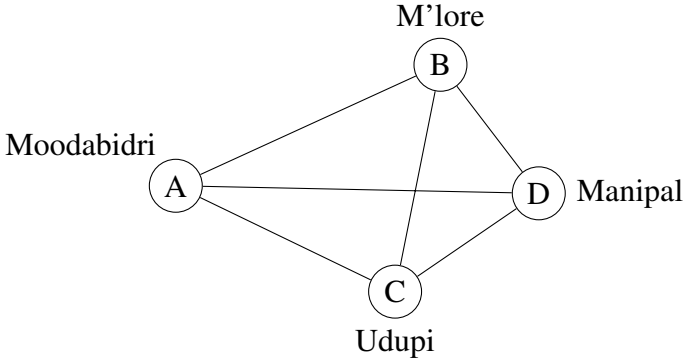


Fig: Graph

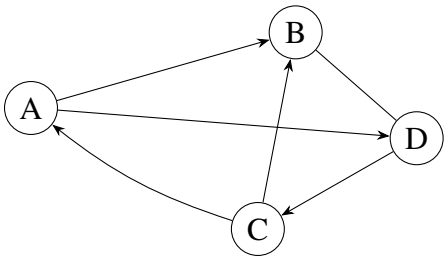


Fig: Directed Graph.

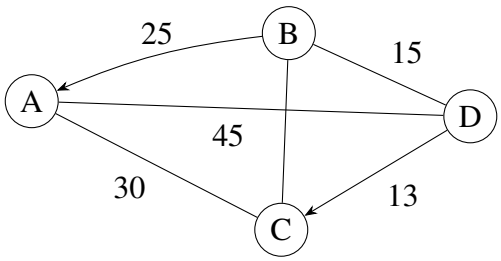


Fig: Weighted graph.

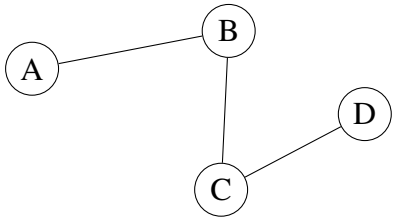


Fig: Graph and also
a tree.
(Because no loops).

Abstract Data Type: (ADT):

ADT refers to a set of data values and associated operations that are specified accurately, independent of any particular implementation. With an ADT, we know what a specific data type can do, but how it actually does it is hidden. In broader terms, the ADT consists of a set of definition that allow us to use the functions while hiding the implementation.

An ADT can be further defined as a data declaration packaged together with the operations that are meaningful for the data type. In other words, we encapsulate the data and the operations on the data, and then we hide them from the user.

Abstract Data Type Model.

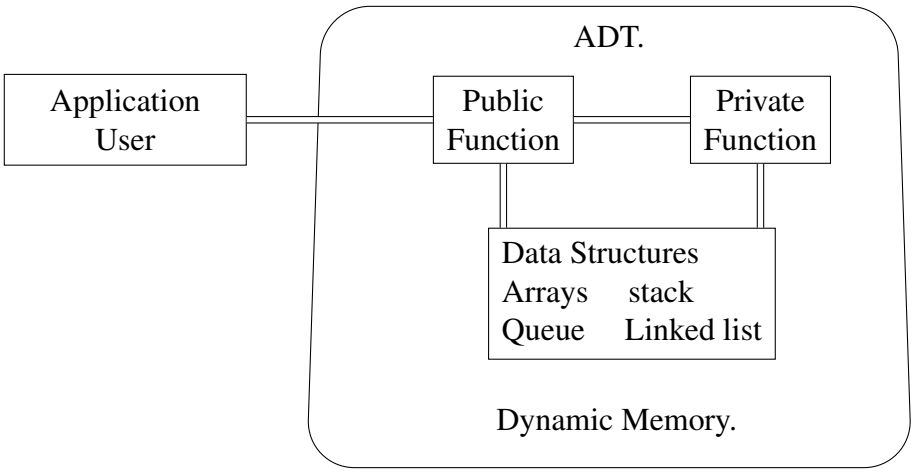


Fig: ADT Model.

There are two different parts in ADT Model.

* Functions
  - Public Functions
  - Private Functions.
* Data Structures.

Both parts are contained within ADT model itself, and do not come within the scope of the application program. On the other hand, data structures are available to all of the ADTs functions as required and a function may call on any other function to accomplish its task. This means that data structure and functions are within the scope of each other.

Data are entered, accessed, modified and deleted through the external application programming interface. This interface can only access the public functions. For each ADT operation, there is an algorithm that performs its specific task. The operation name and parameters are available to the application, and they provide the only interface to the application.

ADT model can be implemented using all available data structures. Two basic data structures, namely array and linked list are the main types used in ADT.

ADT: Array.

ADT Array is

objects: A set of pairs <index, value> where for each value of index there is a value from the set item. Index is a finite ordered set of one (or) more dimensions.
Ex. {0, 1, 2, ... n-1} – One Dimension
{(0,0) (0,1), (0,2) ... (n,m)} – Two Dimension

Functions:

for all A ∈ Arrays, i ∈ index, x ∈ items
j, size ∈ integer.

ArrayCreate(j, list) ::= returns an array of j dimensions where list is a j tuple whose ith element is the size of ith dimension. Items are undefined

ItemRetrive(A, i) ::= if (i ∈ index) return the Item associated with index value i in array A
else return error.

ArrayStore(A, i, x) ::= if (i in index)
return an array that is identical to array A except the new pair <i, x> has been inserted
else return error.

Arrays in C:

An array is a collection of similar data elements. These data elements have the same data type. The elements of the array are stored in consecutive memory locations and are referenced by an index (also known as the subscript). The index/Subscript is an ordinal number which is used to identify an element of the array.

Declaration of Arrays (1-D):

Declaring an array means specifying the following:
(An array must be declared before being used):

* Data Type: the kind of value it can store.
  Ex: int, char, float, etc...
* Name: to identify the array (Array Name)
* Size: the maximum number of values that the array can hold.

Syntax: `data_type name[size];`

Ex: int marks[10]
where: – datatype is int
– array name is marks
– Size is 10. (Max 10 int values can be stored).

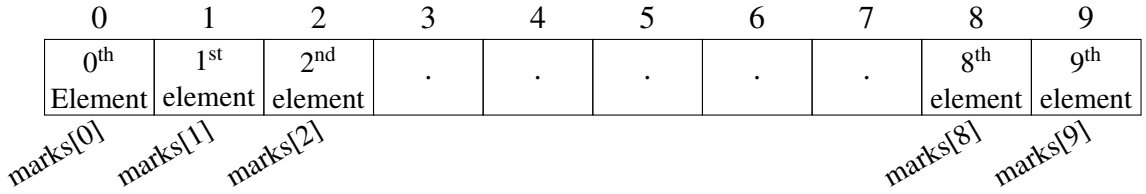


Fig: Memory representation of an array of 10 elements in 1-D.

Calculating the address of array elements (1-D)

In any array, array name is a symbolic reference to the address of the first byte of the array ie base address. When we use the array name, we are actually referring to the first byte of the array.

The subscript/index represents the offset from the beginning of the array to the element being referenced. That is, with just the array name and index, we can calculate the address of any element in the array.

Since an array stores all its data elements in consecutive memory locations, storing just the base address, that is the address of the first element in the array, is sufficient. The address of other elements can simply be calculated using the base address. The formula to perform this calculation is:

Address of data element A[K]

$$\text{Address of A[K]} = \text{Base Address (A)} + \text{Index K} * (\text{Scale factor})$$

ie – for int scale factor is 2 (or) 4 bytes
– for float – 4 bytes
– for char – 1 byte

Ex: int mark[10] = {99, 67, 82, 45, 22, 91, 50, 62, 34, 77}
Assume Base Address is 4000 and Size of int is 2 bytes

Index:	0	1	2	3	4	5	6	7	8	9
Value:	99	67	82	45	22	91	50	62	34	77
Address	4000	4002	4004	4006	4008	4010	4012	4014	4016	4018.

marks[4]: Address can be calculated as
= Base Address + 2 ∗ scale factor
= 4000 + 2 ∗ 4 = 4008.

Operation on Arrays:

There are no of operations that can be performed on arrays. These operations include:

- ∗ Inserting an element in an array
- ∗ Deleting an element from an array
- ∗ Updating an element in an array.
- ∗ Searching an element in an array
- ∗ Sorting an array in ascending/descending order.
- ∗ Traversing (display) an array
- ∗ Merging two arrays, etc. . .

Pgm. Write a C program to read n array element and display the array content (Normal and Pointer approach)

```
#include<stdio.h>
int main()
{ int a[50], n, i;
  printf("Enter the size of
          array \n");
  scanf("%d", &n);
  printf("Enter array elements\n");
  for(i=0; i<n; i++)
     scanf("%d", &a[i]);
  printf("Display array
          elements\n");
  for(i=0; i<n; i++)
     printf("%d\t", a[i]);
  printf("\n");
  return 0;
} /* End of Program */
```

Pgm: Normal Approach

```
#include<stdio.h>
int main()
{ int a[50], n, i;
  printf("Enter the size of
          array \n");
  scanf("%d", &n);
  printf("Enter array elements\n");
  for(i=0; i<n; i++)
     scanf("%d", (a+i));
  printf("Display array element
          \n");
  for(i=0; i<n; i++)
     printf("%d\t", *(a+i));
  printf("\n");
  return 0;
} /* End of Program */
```

Pgm: Pointer Approach.

Pgm: Write a ‘C’ program to perform below operation on an array.

- ∗ Insert an element in an array (location)
- ∗ Delete an element from an array (location).
- ∗ Update an element in an array (w.r.t location).
- ∗ Display the content of an array

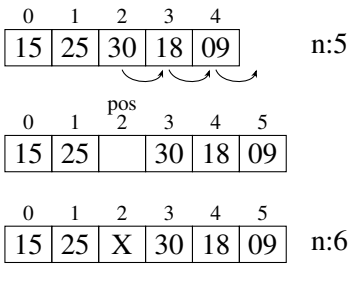
```
#include<stdio.h>
int main()
{
    int array_insert(int a[], int n, int pos, int value);
    int array_delete(int a[], int n, int pos);
    void array_update(int a[], int pos, int value);
    void array_display(int a[], int n);
    int a[100], i, n, pos, value, choice;

    printf("Enter size of array \n");
    scanf("%d", &n);
    printf("Enter array elements \n");
    for(i=0; i<n; i++)
        scanf("%d", &a[i]);
    do
    { printf("Enter your choice of operation: \n");
      printf("1. Insert, 2.delete, 3-Update, 4-Display,
              any other number to exit \n");
      scanf("%d", &choice);
      switch(choice)
      {
        case 1: printf("Enter position and value to
                        be inserted \n");
                scanf("%d%d", &pos, &value);
                if (pos<n)
                    n=array_insert(a, n, pos, value);
                else
                    printf("Position is out of array range \n");
                break;
        case 2: printf("Enter position of the element to
                        be deleted \n");
                scanf("%d", &pos);
                if (pos<n)
                    n= array_delete(a, n, pos);
                else
                    printf("Position is out of array range \n");
                break;
        case 3: printf("Enter position and value to be
                        updated \n");
                scanf("%d%d", &pos, &value);
                if (pos<n)
                    array_update(a, pos, value);
                else
                    printf("Position is out of array range\n");
                break;
        case 4: printf("Display the content of array \n");
                array_display(a, n);
                break;
        default: printf("Invalid choice, EXIT \n");
      } /* End of switch statement */
    } while (choice >=1 && choice <=4);
      /* End of DO-while statement */

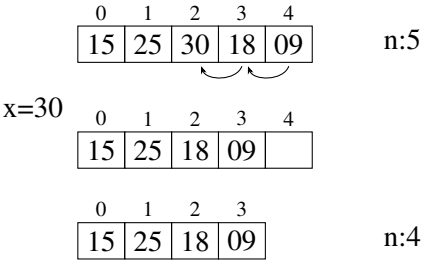
    return 0;
```

```
  } /* End of program */
```

```
int array_insert(int a[], int n, int pos, int
   value)
{ int i;
  for(i=n-1; i>=pos; i--)
      a[i+1] = a[i];
  a[pos] = value;
  n= n+1;
  return (n);
} /* End of array-insert */
```




```
int array_delete(int a[], int n, int pos)
{
  int value;
  value = a[pos];
  printf("Element deleted from array is %d \n",
          value);
  for(i=pos; i<n-1; i++)
      a[i] = a[i+1];
  n=n-1;
  return (n);
} /* End of array-delete */
```




```
  void array_update(int a[], int pos, int value)
  {
    printf("Element present at %d position before
            update is %d \n", pos, a[pos]);

    a[pos] = value;
    printf("Element present at %d position after
            update is %d \n", pos, a[pos]);

    return;
  } /* End of array-update */

  void array_display(int a[], int n)
  { int i;
    for(i=0; i<n; i++)
        printf("%d \t", a[i]);
    printf("\n");
    return;
  } /* End of array-display */
```

---

* ASSIGNMENT *

Pgm: Write C program to find a Key element using linear Search. (use 'c' function)

Pgm: Write C program to find/search a Key element using Binary Search. (use 'c' function)

Pgm: Write C program to sort n elements using Bubble Sort. (use 'c' function)

Pgm: Write C program to sort n elements using Selection Sort. (use 'c' function).

Representation of Polynomials using arrays:

Polynomials is an expression of more than two algebraic terms, especially the sum of several terms that contain different powers of the same variable(s).

This can be represented has

$a.x^e$

where a: co-efficient
e: exponent and
x: variable.

Ex: $A(x) = 2x^3 + 4x^2 + 5x + 3$
Degree of A(x) = 3.

The largest exponent of a polynomial is called degree

A 1-D array is used for representing a single variable polynomial. The index of such an array can be considered as an exponent and the co-efficient can be stored at that particular index.

Ex: $3x^4 + 5x^3 + 6x^2 + 10x + 4$

This exp can be stored in 1-D array as:

Exponents →	0	1	2	3	4
Co-efficient →	4	10	6	5	3

So the degree of this expression is 4 and size of this polynomial array is 5. Exponent and co-efficient are stored at 0, 1, 2, 3, 4 indexes.

Pgm: Write c program to demonstrate how 1-D arrays can be used for polynomial storage and addition.
Note: Use dynamic memory allocation to create a resultant polynomial expression.

```
#include<stdio.h>
#include<stdlib.h>
int main()
{ void print_poly(int p[], int pdegree);
  void add_poly(int p1[], int p1degree,
                int p2[], int p2degree,
                int *p3, int p3degree);
  int p1[] = {5, 2, 0, 3}, p1degree=3;
  int p2[] = {1, 6, 5, 0, 4}, p2degree=4;


  int *p3, p3degree;

  printf("\nPolynomial 1 = ");
  print_poly(p1, p1degree);

  printf("\nPolynomial 2 = ");
  print_poly(p2, p2degree);

  p3degree = (p1degree > p2degree)? p1degree:
                                    p2degree;
  p3 = (int *)malloc(sizeof(int)*p3degree+1);
  add_poly(p1, p1degree, p2, p2degree, p3, p3degree);
```

$3x^3 + 2x + 5$

$4x^4 + 5x^2 + 6x + 1$

```
  printf("\n Addition of polynomial 1+2 = ");
  print_poly(p3, p3degree);
  printf("\n");
  free(p3);
  return 0;
} /* End of main */
```

p3 = p1 + p2;
= $4x^4 + 3x^3 + 5x^2$
+ 8x + 6

```
void print_poly(int p[], int pdegree)
{ int i;
  for(i=pdegree; i>=0; i--)
  { if (p[i]!=0)
    { if (i==pdegree)
         printf("%dx^%d", p[i], i);
      else if (i==0)
         printf(" + %d", p[i]);
      else
         printf(" + %dx^%d", p[i], i);
    }
  }
} /* End of print poly */

void add_poly(int p1[], int p1degree,
              int p2[], int p2degree,
              int p3[], int p3degree)
{ int i;
  for(i=p3degree; i>=0; i--)
      p3[i] = 0;
  for(i=p1degree; i>=0; i--)
      p3[i] = p3[i] + p1[i];
  for(i=p2degree; i>=0; i--)
      p3[i] = p3[i] + p2[i];
  return;
} /* End of add poly */
```

Two-Dimensional Arrays (2-D)

A 2-D array is specified using two subscripts where the first subscript denotes the ROW and the second denotes the COLUMN. Below figure shows a 2-D array, which can be treated as a matrix (or) a table.

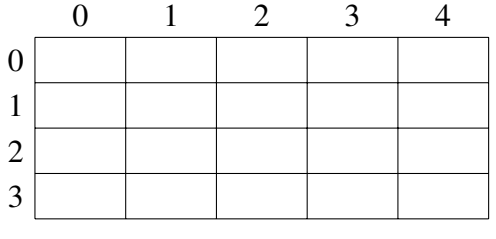


Fig: 2-D array with 4 Rows and 5 column.

Declaring 2-D arrays:

The declaration statement tells the compiler the name of the array, the datatype of each element in the array, and the size of each dimension (Row, column).

2-D array is declared as:

```
data_type array_name[row_size][col_size];
```

Ex: int mat[3][5];
where: data type is int
Array name is mat
row size is 3 and (Max 3 rows with 5 colums)
col size is 5. (Max 5 colums in each row).

From the above example, the 2-D array 'mat' is a 3X5 order matrix/table. The first element of the array is denoted by mat[0][0], the second element as mat[0][1] and so on. The pictorial form of a 2-D array is shown as:

	col-0	col-1	col-2	col-3	col-4
Row-0	mat[0][0]	[0][1]	[0][2]	[0][3]	[0][4]
Row-1	[1][0]	[1][1]	[1][2]	[1][3]	[1][4]
Row-2	[2][0]	[2][1]	[2][2]	[2][3]	[2][4]

3X5

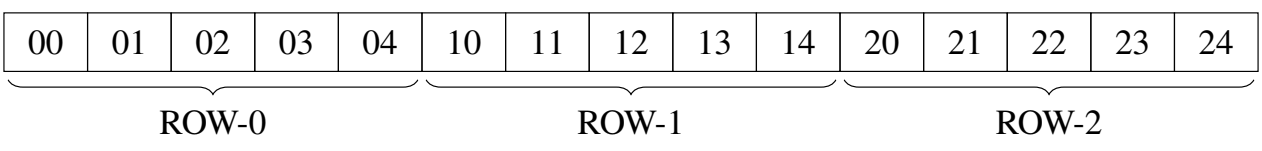
Although above table of 2-D array, in the memory those elements actually will be stored sequentially.

There are two ways of storing a 2-D arrays in the memory.

1. Row Major Order &
2. Column Major order.

Row major order:

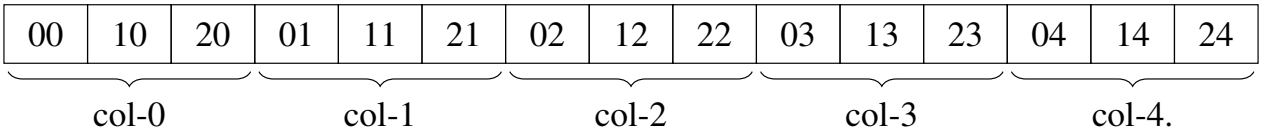
The elements of the first row are stored before the elements of the second and third rows. ie, the elements of the array are stored row by row where first row 'n' elements will occupy the first 'n' locations.



3X5

Column major order:

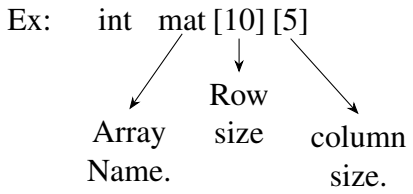
In this case, the elements of the first column are stored before the elements of the second column & 3rd column. ie. the elements of the array are stored column by column where first column 'm' elements will occupy the first 'm' locations.



3X5.

Calculating the address of 2-D array elements.

In any type of arrays (1-D (or) 2-D (or) multi-D) the compiler does not keep track of the address of every element in the array. It stores only the address of the first element and calculates the address of other elements from the base-address (address of the first byte of the array).



If the array elements are stored in Row Major order, then address is calculated as:

address[i][j] = Base Address (i ∗ column size + j) ∗ Scale factor (SF)

where i: Row value
j = col value,

Ex:

mat[1][2] = what is address loc of [1][2].

Assume Base Address is 2000, then address can be calculated as:

= Base Address (Row value ∗ col size + col value) ∗ SF

= 2000 (1 ∗ 5 + 2) ∗ 4 | if int take 4 bytes

= 2000 (7) ∗ 4

= 2028.

Ex: int mat[3][5] = {{5, 7, 9, 4, 2}, {51, 71, 92, 14, 12},
{12, 14, 99, 82, 32}};

the above initialized 2-D array in memory with value can be shown as ((row, col), Address and value):

	col-0	col-1	col-2	col-3	col-4
Row-0	0,0 (row, col) 2000 (address) (value) 5	0,1 2004 7	0,2 2008 9	0,3 2012 4	0,4 2016 2
Row-1	1,0 2010 51	1,1 2024 71	1,2 2028 92	1,3 2032 14	1,4 2036 12
Row-2	2,0 2040 12	2,1 2044 14	2,2 2046 99	2,3 2052 82	2,4 2056. 32

3X5

Pgm: W.a.'c' program to read two matrices, find the product and display the result using functions.

```
#include<stdio.h>
#include<stdlib.h>
int main()
{ void read_mat(int mat[][10], int m, int n);
  void print_mat(int mat[][10], int m, int n);
  void product_mat(int mat1[][10], int m, int n,
                   int mat2[][10], int p, int q,
                   int mat3[][10]);
  int m, n, p, q, mat1[10][10], mat2[10][10], mat3[10]
                                                  [10];

  printf("Enter order of Matrix-1 RxC \n");
  scanf("%d%d", &m, &n);
  printf("Enter order of Matrix-2 RxC \n");
  scanf("%d%d", &p, &q);
  if (n!=p)
  { printf("Order of the matrices are
           incorrect, exit the program \n");
    exit(0);
  }
  printf("Enter Matrix-1 elements \n");
  read_mat(mat1, m, n);
  printf("Enter Matrix-2 elements \n");
  read_mat(mat2, p, q);
  product_mat(mat1, m, n, mat2, p, q, mat3);
  printf("Matrix1 elements are \n");
  print_mat(mat1, m, n);
  printf("Matrix2 elements are \n");
  print_mat(mat2, p, q);
  printf("Product of Mat1 x Mat2 is \n");
  print_mat(mat3, m, q);
  return 0;
} /* End of main program */

void read_mat(int mat[][10], int m, int n)
{ int i, j;
  for(i=0; i<m; i++)
     for(j=0; j<n; j++)
        scanf("%d", &a[i][j]);
```

```
  return;
} /* End of read matrix */

void print_mat(int mat[][10], int m, int n)
{ int i, j;
  for(i=0; i<m; i++)
  { for(j=0; j<n; j++)
       printf("%d \t", a[i][j]);
    printf("\n");
  }
  return;
} /* End of print matrix */

void product_mat(int mat1[][10], int m, int n,
                 int mat2[][10], int p, int q,
                 int mat3[][10])
{ int i, j, k;
  for(i=0; i<m; i++)
     for(j=0; j<q; j++)
     { mat3[i][j] = 0;
       for(k=0; k<n; k++)
          mat3[i][j] = mat3[i][j] +
                       mat1[i][k] * mat2[k][j];
     }
  return;
} /* End of product mat */
```

Sparse Matrix:

A sparse matrix is a matrix that has very few non-zero elements.

Ex:

$$\begin{bmatrix} 2 & 0 & 0 & 0 \\ 4 & 0 & 0 & -3 \\ 0 & 0 & 0 & 0 \\ 8 & 0 & 0 & 1 \\ 0 & 0 & 6 & 0 \end{bmatrix}_{5\times4} \qquad \begin{bmatrix} 4 & 0 & 0 & 0 & 0 \\ 3 & -5 & 0 & 0 & 0 \\ 0 & 0 & 6 & 0 & 0 \\ -7 & 8 & 3 & 3 & 0 \\ 5 & -2 & 0 & 2 & 8 \end{bmatrix}_{5\times5} \qquad \begin{bmatrix} 4 & 5 & 0 & 0 & 0 \\ 8 & 0 & 8 & 0 & 0 \\ 0 & 7 & -3 & 2 & 0 \\ 0 & 0 & 9 & 0 & 1 \\ 0 & 0 & 0 & -2 & 7 \end{bmatrix}_{5\times5}$$

(i) Sparse Matrix

(ii) Triangular Sparse Matrix (Lower Sparse Matrix)

(iii) Tridiagonal Sparse Matrix

The general types of sparse matrices, which occurs in various applications are shown in above figures. The fig (i) is the collection of 5X4 order matrix with more zero (0) elements, fig (ii) where all entries above the main diagonal are zero (0) (or) equivalently, where non-zero entries can only occurs on (or) below the main diagonal, is called triangular matrix. This is also a example for lower triangular matrix. If the non-zero entries occurs on (or) above the main diagonal is called upper triangular matrix. Fig (iii) where no-zero entries can only occur on the diagonal (or) on elements immediately above (or) below the diagonal is called a tridiagonal matrix.

The natural method of representing matrices in memory as 2-D arrays may not be suitable for Sparse matrices. ie. one may save space by storing only those entries which may be non-zero.

Sparse Matrix Representation:

By examining any matrix, we can characterize uniquely any element within a matrix by using the triple <row, col, value>. This means that we can use an array of triples to represent a sparse matrix. We should organize the triples so that the row indices are in ascending order. We can go one step further by also requiring that all the triples for any row be stored so that the column indices are in ascending order. In addition, to ensure that the operations terminate, we must know the number of rows, columns and the number of non-zero elements in the matrix. Putting all this information together, we can implement sparse matrix representation as:

```
#define MAX 101 /* Maximum element + 1 */
typedef struct
     { int row;
       int col;
       int value;
     } term;
term a[MAX];
```

where a is an array

a[0].row : contains the no of rows
a[0].col : contains the no of columns.
a[0].value : contains the total no of nonzero elements.
a[1].row, a[1].col, a[1].value } from 1st position, start storing non-zero element triples representation ie, row, col and value

row = Row index is stored in the field row
col = Column index is stored in the field col
value = Value is the field of actual element

Ex:

	col-0	col-1	col-2	col-3	col-4	col-5
Row-0	15	0	0	22	0	−15
Row-1	0	11	3	0	0	0
Row-2	0	0	0	−6	0	0
Row-3	0	0	0	0	0	0
Row-4	91	0	0	0	0	0
Row-5	0	0	28	0	0	0

6×6

Matrix with few non-zero elements.

Sparse matrix of the given matrix, which is generated in triples format is:

	row	col	value
a[0]	6	6	8.
a[1]	0	0	15
a[2]	0	3	22
a[3]	0	5	-15
a[4]	1	1	11
a[5]	1	2	3
a[6]	2	3	-6
a[7]	4	0	91
a[8]	5	2	28.

Sparse Matrix
(stored as triples)- Row



	row	col	value
t(0)	6	6	8
t(1)	0	0	15
t(2)	0	4	91
t(3)	1	1	11
t(4)	2	1	3
t(5)	2	5	28
t(6)	3	0	22
t(7)	3	2	-6
t(8).	5	0	-15

Transposed Sparse Matrix
COL- (stored as triples).

Function to perform Transpose of a Sparse Matrix

```
void transpose_sparse(term s[], term t[])
{ int n, i, j, t_count;
  n = a[0].value;          /* n total no of values */
  t[0].row = s[0].col;     /* s is a sparse mat */
  t[0].col = s[0].row;     /* t is transpose mat */
  t[0].value = n;
  if (n>0)
  { t_count = 1;
    for(i=0; i<a[0].col; i++)
      for(j=1; j<=n; j++)
      {
        if (s[j].col == i)
        { t[t_count].row = s[j].col;
          t[t_count].col = s[j].row;
          t[t_count].value = s[j].value;
          t_count++;
        }
      }
  }
} /* End of transpose sparse */
```

Pgm: Write ‘c’ Program to read a sparse matrix and convert it into triples representation (1-D array). Later transpose sparse matrix.

```
#include<stdio.h>
typedef struct
    { int row;
      int col;
      int value;
    }sparse;
int main()
{ void generate_sparse(int a[][10], int m, int n, sparse s[]);
  void transpose_sparse(sparse s[], sparse t[]);
  void print_sparse(sparse s[]);
  sparse s[100], t[100];
  int a[10][10], m, n, i, j;
```

```
  printf("Enter order of input matrix R x C \n");
  scanf("%d%d", &m, &n);
  printf("Enter matrix with few nonzero
          elements \n");
  for(i=0; i<m; i++)
     for(j=0; j<n; j++)
        scanf("%d", &a[i][j]);
  printf("Entered matrix - sparse\n");
  for(i=0; i<m; i++)
  { for(j=0; j<n; j++)
       printf("%d \t", a[i][j]);
    printf("\n");
  }
  generate_sparse(a, m, n, s);
  printf("Sparse matrix in triples format \n");
  print_sparse(s);
  transpose_sparse(s, t);
  printf("Sparse matrix - Transpose format \n");
  print_sparse(t);
  return 1;
} /* End of Main Pgm */

void print_sparse(sparse s[])
{ int i;
  printf("ROW      COL      VALUE \n");
  for(i=0; i<=s[0].value; i++)
     printf("%d \t %d \t %d \t\n", s[i].row, s[i].col,
                                    s[i].value);
  printf("\n");
  return;
} /* End of print-sparse */

void generate_sparse(int a[][10], int m, int n,
                     sparse s[])
{ int k, i, j;
  k=0;
  s[0].row = m;
  s[0].col = n;
  for(i=0; i<m; i++)
     for(j=0; j<n; j++)
     { if(a[i][j]!=0)
       { k++;
         s[k].row = i;
         s[k].col = j;
         s[k].value = a[i][j];
       }
     }
  s[0].value = k;
  return;
} /* End of generate-sparse */

void transpose_sparse(sparse s[], sparse t[])
{ int n, count, i, j;
```

```
  n= s[0].value;
  t[0].row = s[0].col;
  t[0].col = s[0].row;
  t[0].value = s[0].value;
  count =0;
  for(i=0; i<s[0].col; i++)
     for(j=1; j<=n; j++)
     { if (s[j].col == i)
       {
         count++;
         t[count].row = s[j].col;
         t[count].col = s[j].row;
         t[count].value = s[j].value;
       }
     }
  return;
} /* End of transpose-sparse */.
```

Advantages of Arrays in C

* Easy Access:
  Elements can be accessed easily using its index
* Efficient Memory Access:
  All elements in arrays are stored in contiguous (sequential) memory locations. which helps to improve traversal faster
* Simplicity:
  Arrays are simple to declare and use compared to more complex DS like linked list and trees.
* Easier sorting & Searching.
  Since elements are stored sequentially, sorting and searching algorithms can be easily implemented.
  Ex: Binary/linear Search, Bubble, selection, sorting.
* Static memory allocation:
  Fixed array size at compile time, which is useful in memory constrained environments.

Disadvantage of Arrays in C:

* Fixed Size:
  Size of an array must be known at compile time. Once defined it cannot grow (or) shrink.
* Inefficient Insertion & Deletion:
  Inserting (or) deleting an element requires shifting of elements
* Memory Wastage:
  If the declared array size is larger than needed, unused elements waste memory.
* Homogeneous (same) elements only.
  Arrays can only store elements of one datatype.
* Static Nature:

User cannot easily perform dynamic operations like resizing (or) reallocation without using pointers (or) dynamic memory (malloc, calloc).

———————*———————*———————

Question Bank

M: Marks, L: RBT (Revised Bloom's Taxonomy) level, C: Course Outcome.

Introduction and Classification of Data Structures

Q. No.	Question	M	L	C
1	Define a data structure. Differentiate between primitive and non-primitive data structures with examples.	06	L2	CO1
2	With a neat block diagram, describe how data structures are classified. Give two examples for each category.	08	L2	CO1
3	Distinguish between linear and non-linear data structures. Why are arrays and linked lists called linear even though their memory storage need not be sequential?	06	L2	CO1
4	Explain the stack (LIFO) and the queue (FIFO) using real-life analogies. Draw a neat diagram of each and list the operations performed on them.	08	L2	CO1
5	What is a linked list? Explain the structure of a node and draw a linked list with four nodes, showing the external pointer and the NULL pointer. List the operations performed on a linked list.	08	L2	CO1
6	Explain how a tree represents hierarchical data. Represent a student record (Reg No, Name, Address, Branch, Sex) as a tree.	06	L2	CO1
7	Define a graph. With suitable diagrams, illustrate an undirected graph, a directed graph, a weighted graph and a graph that is also a tree.	08	L2	CO1

Abstract Data Type (ADT)

Q. No.	Question	M	L	C
8	What is an Abstract Data Type? Draw the ADT model and explain the role of public functions, private functions and data structures in it.	08	L2	CO1
9	Write the ADT specification of a one-dimensional array, giving its objects and the functions ArrayCreate, ItemRetrive and ArrayStore. Illustrate each function with an example.	08	L3	CO1
10	"With an ADT, we know what a data type can do, but not how it does it." Justify this statement with an example.	05	L2	CO1

Arrays in C (1-D)

Q. No.	Question	M	L	C
11	Define an array. Explain the declaration of a 1-D array in C with its syntax, and draw the memory representation of `int marks[10]`.	06	L2	CO1
12	Write the formula to calculate the address of A[K] in a 1-D array. For `float b[20]` with base address 1000 and a scale factor of 4 bytes, find the addresses of b[5] and b[12].	06	L3	CO1
13	List and briefly explain the operations that can be performed on arrays.	06	L2	CO1
14	Write a C program that reads n integers into an array and prints them using pointer notation. Explain how `*(a+i)` is related to `a[i]`.	08	L3	CO2
15	Write a menu-based C program with separate functions to (i) add a new element at a specified position, (ii) remove the element at a specified position and (iii) print all the elements of an array.	09	L3	CO2
16	Write a C function that replaces the element at a given position of an array with a new value, displaying the element before and after the change.	05	L3	CO2
17	Write a C program using a function to search for a key in an array using linear search. Display the position if found, else a suitable message.	08	L3	CO2
18	Implement binary search as a C function that returns the index of a key in a sorted array (or -1 if absent). Trace the search for key 45 in $\{5, 12, 19, 27, 33, 45, 51, 68\}$.	09	L3	CO2
19	Write a C program using a function to arrange n numbers in ascending order using bubble sort. Show the array after each pass for $\{29, 10, 14, 37, 13\}$.	08	L3	CO2
20	Write a C program using a function to sort n elements in ascending order using selection sort. Trace it for $\{64, 25, 12, 22, 11\}$.	08	L3	CO2
21	Write a C program to merge two arrays into a third array and display the merged array.	06	L3	CO2

Representation of Polynomials using Arrays

Q. No.	Question	M	L	C
22	Explain how a single-variable polynomial is stored in a 1-D array. Represent $7x^5 + 2x^3 - 4x + 9$ in an array and state its degree and the array size.	06	L3	CO1
23	Write a C program using functions to store two polynomials in 1-D arrays, add them and print the result. Use dynamic memory allocation for the resultant polynomial.	09	L3	CO2

Q. No.	Question	M	L	C
24	Given P1(x) = $4x^3 + 3x + 2$ and P2(x) = $5x^4 + x^3 + 6x^2 + 1$, show their array representations and the array of P1 + P2.	05	L3	CO2

Two-Dimensional Arrays

Q. No.	Question	M	L	C
25	How is a 2-D array declared in C? With an example such as `int mat[3][4]`, explain how it is viewed as a table and how it is actually laid out in memory.	08	L2	CO1
26	Explain row major order and column major order of storing a 2-D array, with neat diagrams for a 3×5 array.	08	L2	CO1
27	Write the formula for the address of mat[i][j] in row major order. For `int mat[4][6]` with base address 3000, where int takes 4 bytes, compute the address of mat[2][3] and mat[3][5].	06	L3	CO1
28	Write a C program using separate functions to read two matrices of the same order, add them and display the resultant matrix.	09	L3	CO2
29	Write a C program using functions to multiply two matrices. The program must check whether the matrices can be multiplied and display a message otherwise.	09	L3	CO2

Sparse Matrices

Q. No.	Question	M	L	C
30	What is a sparse matrix? Explain the general sparse matrix, lower and upper triangular matrices and the tridiagonal matrix with an example of each.	08	L2	CO1
31	Why is a normal 2-D array not suitable for storing a sparse matrix? Explain the triple <row, col, value> representation, giving the structure definition and the meaning of a[0].row, a[0].col and a[0].value.	06	L2	CO1
32	Represent the following matrix using triples and obtain the triples of its transpose. Show both tables. $\begin{bmatrix} 0 & 6 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & -3 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 9 & 0 & 0 & 0 & 0 & 2 \\ 0 & 0 & 4 & 0 & 0 & 0 \end{bmatrix}$	09	L3	CO2

Q. No.	Question	M	L	C
33	Write a C function that reads a matrix and generates its sparse matrix representation in triple form, and a function to print the triples.	08	L3	CO2
34	Write a C function to transpose a sparse matrix stored as triples. Explain how the function keeps the transposed triples in row order.	08	L3	CO2

Advantages and Disadvantages of Arrays

Q. No.	Question	M	L	C
35	Discuss the advantages and disadvantages of arrays in C.	06	L2	CO1
36	Arrays use static memory allocation. Explain how this is both an advantage and a disadvantage.	04	L2	CO1

Stacks, Queues, Expressions and Recursion

Q. No.	Question	M	L	C
37	Explain recursion, identifying the base case and the recursive case. Write a recursive C function for the Tower of Hanoi and list the moves needed to transfer three disks from peg A to peg C using peg B as auxiliary.	08	L3	CO1
38	Define a stack. Implement PUSH, POP and DISPLAY on a stack using a static array in C, displaying proper messages for stack overflow and underflow.	08	L3	CO2
39	Evaluate the following postfix expressions using a stack, showing the stack contents after each symbol (^ : power, % : modulus): (i) 6 3 2 ^ * 4 2 / – 5 + (ii) 8 2 3 * – 4 2 % +	08	L3	CO2
40	Define an ordinary queue. Write a C program to insert, delete and display elements of a queue implemented with a static array, with suitable messages for overflow and underflow.	08	L3	CO2
41	Convert the following infix expressions to postfix using a stack, showing the stack contents at each step: (i) (A * B + C / (D – E)) ^ F (ii) (4 + 8) * (6 – 5) / ((3 – 2) * (2 + 2))	08	L3	CO2

********** E N D **********